

LAPORAN PRAKTIKUM SISTEM OPERASI

MODUL 14 SCRIPTING



Disusun Oleh :

Ilham Lii Assidaq 2311104068

Dosen Pengampu :

Yudha Islami Sulistya, S.Kom., M.Cs

PROGRAM STUDI S1 SOFTWARE ENGINEERING

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

1. Dasar Teori

A. Pengantar Bash Scripting di Ekosistem Linux

Selain lewat tampilan visual atau grafis, pengguna Linux kerap memanfaatkan command-line interface (CLI) untuk mengoperasikan sistem. Di sinilah shell berfungsi sebagai jembatan utama yang menghubungkan instruksi dari pengguna langsung ke dalam kernel sistem operasi. Teknik penulisan rangkaian perintah otomatis menggunakan shell inilah yang disebut dengan bash scripting. Menguasai keterampilan ini menjadi kompetensi fundamental yang sangat krusial, terutama bagi para mahasiswa ilmu komputer, pengembang perangkat lunak, hingga administrator jaringan.

B. Memahami Shell dan Karakteristik Bash

Secara prinsip, shell merupakan sebuah program penerjemah yang bertugas menangkap perintah pengguna, mengolahnya, lalu mengirimkannya ke kernel agar dieksekusi. Di lingkungan Linux, terdapat beragam variasi shell seperti sh, zsh, fish, dan bash. Di antara semuanya, Bourne Again Shell (Bash) mendominasi sebagai standar bawaan di mayoritas distro Linux karena mendukung penulisan skrip, sehingga sekumpulan instruksi rumit dapat disimpan dalam satu berkas ber ekstensi khusus untuk dijalankan secara otomatis kapan saja.

C. Fondasi Utama Berkas Bash Script

- Shebang: Setiap kode skrip Bash wajib diawali dengan baris kode `#!/bin/bash` pada bagian paling atas. Kode ini menjadi penanda bagi sistem operasi untuk membaca dan mengeksekusi berkas menggunakan interpreter Bash.
- Izin Operasional (Eksekusi): Berkas skrip tidak akan bisa berjalan sebelum diberikan hak akses eksekusi oleh sistem. Pengguna perlu mengonfigurasinya terlebih dahulu melalui perintah `chmod +x nama_skrip.sh`.
- Pengeksekusian: Setelah izin diberikan, skrip tersebut dapat langsung dijalankan di dalam terminal dengan mengetikkan perintah `./nama_skrip.sh`.

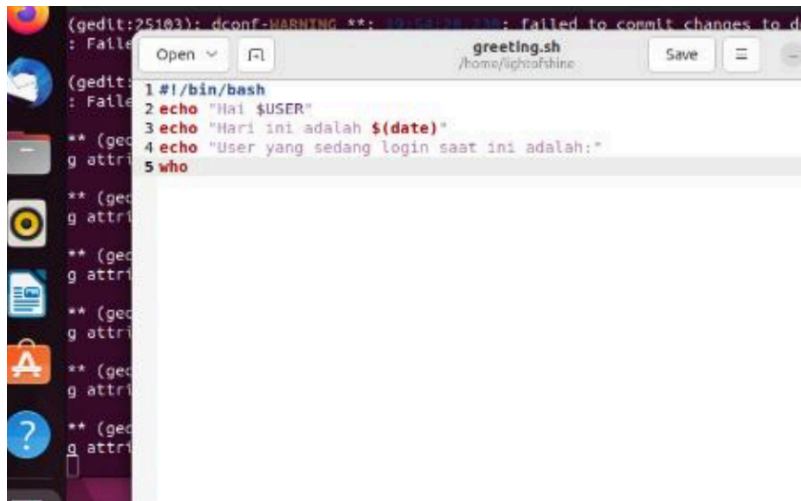
2. Manfaat

Melalui praktikum pada modul ini, manfaat yang saya dapatkan adalah mampu menguasai teknik penulisan bash scripting untuk mengotomatisasi berbagai perintah operasional di lingkungan sistem operasi Linux. Saya juga memperoleh pemahaman mendalam mengenai peran penting shell sebagai penerjemah instruksi antara pengguna dan kernel sistem.

3. Jurnal

A. 1. [15 Poin] PERMULAAN

- a. Buatlah file bernama greeting.sh sesuai dengan template code.
- b. Buatlah script pada greeting.sh sehingga:
 - i. Dapat menyapa user
 - ii. Menampilkan tanggal hari ini
 - iii. Menampilkan user yang sedang login saat iniContoh eksekusi:



```
(gedit:25103): dconf-WARNING **: ... : failed to commit changes to dconf
(gedit:25103): Failed to open file /home/ligheafshine/.greeting.sh: No such file or directory
(gedit:25103): Failed to open file /home/ligheafshine/.greeting.sh: No such file or directory
** (gedit:25103): g attri
1 #!/bin/bash
2 echo "Hai $USER"
3 echo "Hari ini adalah $(date)"
4 echo "User yang sedang login saat ini adalah:"
5 who
```

B. [15 Poin] PENGONDISIAN (WHILE)

- a. Buatlah file bernama greeting_1.sh sesuai dengan template code.

b. Buatlah script pada `greeting_1.sh` sehingga dapat menampilkan “selamat pagi” pada pagi hari (05:01-10:00), “selamat siang” pada siang hari (10:01-15:00), “selamat sore” pada sore hari (15:01-19:00) “selamat malam” pada malam hari (19:01-05:00). Contoh eksekusi:

```
$/greeting_1.sh  
Sekarang jam 13  
Selamat siang User
```

C. [15 Poin] PERULANGAN a. Buatlah file bernama `countdown.sh` sesuai dengan template code. b. Buatlah script sehingga menghasilkan countdown dimulai dari angka 10 hingga angka 1 lalu diikuti tulisan “GO!”. Contoh eksekusi:

```
$/countdown.sh  
10  
9  
8  
7  
6  
5  
4  
3  
2  
1  
GO!
```



D. [15 Poin] INPUT PENGGUNA a. Buatlah file bernama `countdown_1.sh` sesuai dengan template code. b. Buatlah script sehingga menghasilkan countdown berdasarkan masukan dari user. Contoh eksekusi:

```
$/countdown_1.sh
Masukkan angka:
9
Mulai countdown!
9
8
7
6
5
4
3
2
1
GO!
```



- E. a. Buatlah file bernama `countdown_2.sh` sesuai dengan template code. b. Buatlah script sehingga menghasilkan countdown berdasarkan parameter script. Pastikan kondisi-kondisi lain ditangani. Contoh eksekusi:

```
$/countdown_2.sh 4
4
3
2
1
GO!

./countdown_2.sh
penggunaan: countdown_2.sh initial_value
```

4. Kesimpulan

saya kini mengerti peran krusial shell sebagai perantara yang menerjemahkan instruksi pengguna ke kernel sistem. Saya juga telah menguasai struktur penting dalam pembuatan skrip, mulai dari penulisan shebang (`#!/bin/bash`) sebagai penentu interpreter, pemberian hak akses eksekusi melalui perintah `chmod +x`, hingga mekanisme pengeksekusian skrip di terminal.

Secara khusus, pembuatan skrip pemrograman seperti berkas `countdown_2.sh` telah mengasah kemampuan logika saya dalam menerapkan otomatisasi perintah Linux menggunakan parameter masukan (input argument). Saya berhasil memahami bagaimana merancang sebuah program berbasis skrip yang mampu memproses kondisi runtunan angka mundur, sekaligus melakukan penanganan kondisi kesalahan (error handling) ketika pengguna tidak menyertakan nilai awal yang dibutuhkan. Pengalaman praktis ini memberikan saya pondasi administrasi sistem yang sangat berguna untuk menunjang efisiensi kerja di bidang pengembangan perangkat lunak kedepannya.